

JAVA PLACEMENT PREPARATION GUIDE

Complete Concept Explanations + Two Program Examples for Every Topic

Topics Covered:

Introduction to Java • JVM Architecture • Program Lifecycle • IDE Setup
Variables • Data Types • Operators • Type Casting • Input Handling
Access Modifiers • final Keyword • Object Class
if-else • switch • for / while / do-while Loops • break & continue • Nested Loops

Prepared for Campus Placements & Technical Interviews

Table of Contents

1. Introduction to Java (History, Features, Editions)
2. JVM Architecture (JVM, JRE, JDK)
3. Program Lifecycle (Compilation & Execution)
4. IDE Setup (Eclipse, IntelliJ)
5. Variables (Declaration & Initialization)
6. Data Types (Primitive & Wrapper)
7. Operators (Arithmetic, Relational, Logical)
8. Type Casting (Widening, Narrowing)
9. Input Handling (Scanner, BufferedReader)
10. Access Modifiers (public, private, protected, default)
11. final Keyword (Variables, Methods, Classes)
12. Object Class (toString(), equals())
13. if-else (Nested Conditions)
14. switch (Traditional & Enhanced)
15. for Loop (Iteration)
16. while Loop (Entry Controlled)
17. do-while Loop (Exit Controlled)
18. break & continue (Flow Control)
19. Nested Loops (Dry Run)

1. Introduction to Java

History

Java was developed by **James Gosling** and his team at Sun Microsystems (now owned by Oracle) in the early 1990s. Originally called **Oak** (named after an oak tree outside Gosling's office), it was renamed Java in 1995. The first public release was Java 1.0 in 1996. Java was designed with the principle of **“Write Once, Run Anywhere” (WORA)** — code compiled on one platform should run on any platform that has a Java Virtual Machine (JVM). Major milestones: Java 1.2 (1998, Platform 2), Java 5 (2004, generics), Java 8 (2014, lambdas & streams), Java 11 (2018, LTS), Java 17 (2021, LTS), and the current long-term support versions.

Features of Java

- **Simple** — Syntax is clean and derived from C/C++ but removes complex features like pointers and multiple inheritance of classes.
- **Object-Oriented** — Everything is an object (except primitives). Supports encapsulation, inheritance, polymorphism, and abstraction.
- **Platform Independent** — Bytecode runs on any OS with a JVM (Windows, Linux, macOS, etc.).
- **Secure** — No explicit pointers, bytecode verification, Security Manager, and sandbox execution.
- **Robust** — Strong memory management (Garbage Collection), exception handling, and type checking.
- **Multithreaded** — Built-in support for concurrent programming.
- **Architecture Neutral** — Bytecode is independent of processor architecture.
- **Portable** — Size of primitive types is fixed across platforms.
- **High Performance** — Just-In-Time (JIT) compiler converts hot bytecode to native code.
- **Distributed** — Supports RMI, networking, and web services out of the box.
- **Dynamic** — Classes can be loaded at runtime; supports reflection.

Editions of Java

Java SE (Standard Edition) — Core platform for desktop and server applications. Contains the essential libraries (java.lang, java.util, java.io, etc.), JVM, and development tools.

Java EE / Jakarta EE (Enterprise Edition) — Built on top of SE for large-scale, multi-tier, distributed enterprise applications. Includes Servlets, JSP, EJB, JPA, JMS, etc.

Java ME (Micro Edition) — For resource-constrained devices such as embedded systems and older mobile phones. Less relevant today with Android (which uses a different runtime).

Example 1: Classic Hello World

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
        System.out.println("Welcome to Java Placement Preparation");
    }
}
// Output:
// Hello, World!
// Welcome to Java Placement Preparation
```

Example 2: Demonstrating Platform Independence Idea

```
public class PlatformInfo {
    public static void main(String[] args) {
        System.out.println("Java Version : " + System.getProperty("java.version"));
    }
}
```

```
        System.out.println("OS Name      : " + System.getProperty("os.name"));
        System.out.println("OS Arch     : " + System.getProperty("os.arch"));
        System.out.println("User Home   : " + System.getProperty("user.home"));
    }
}
// This same .class file runs on Windows, Linux, or macOS without recompilation.
```

2. JVM Architecture (JVM, JRE, JDK)

JDK (Java Development Kit)

JDK is the complete software development kit required to **develop** Java applications. It contains: JRE + development tools (javac compiler, java launcher, javadoc, jar, jdb debugger, etc.). When you install JDK you get everything needed to write, compile, and run Java programs.

JRE (Java Runtime Environment)

JRE is the runtime package required to **run** Java applications. It contains: JVM + core class libraries (rt.jar / modular jars) + supporting files. End users who only need to execute Java programs install JRE. Developers usually install JDK (which already includes JRE).

JVM (Java Virtual Machine)

JVM is an abstract machine that provides a runtime environment in which Java bytecode can be executed. It is the heart of the “Write Once, Run Anywhere” promise. Key components of JVM architecture:

- **Class Loader Subsystem** — Loads, links (verify, prepare, resolve), and initializes .class files.
- **Runtime Data Areas:**
 - Method Area (stores class structures, static variables, constant pool)
 - Heap (objects and arrays – shared among threads)
 - Stack (one per thread – frames with local variables & operand stack)
 - PC Register (address of current instruction)
 - Native Method Stack
- **Execution Engine** — Interpreter + JIT Compiler + Garbage Collector
- **Native Method Interface (JNI)** — Allows Java code to call native (C/C++) libraries
- **Native Method Libraries**

Relationship: $JDK \supset JRE \supset JVM$. You need JDK to develop, JRE to run, and JVM is the engine inside JRE that actually executes bytecode.

Example 1: Checking JDK / JRE / JVM version information

```
public class VersionInfo {
    public static void main(String[] args) {
        // These properties are provided by the JVM
        System.out.println("java.version"      = " + System.getProperty("java.version"));
        System.out.println("java.runtime.version" = " + System.getProperty("java.runtime.version"));
        System.out.println("java.vm.name"       = " + System.getProperty("java.vm.name"));
        System.out.println("java.vm.version"    = " + System.getProperty("java.vm.version"));
        System.out.println("java.home"         = " + System.getProperty("java.home"));
        System.out.println("java.class.path"    = " + System.getProperty("java.class.path"));
    }
}
```

Example 2: Simple illustration of Class Loading & Heap usage

```
class Student {
    String name;
    Student(String name) { this.name = name; }
}

public class JVMDemo {
    public static void main(String[] args) {
        // Objects are created on the Heap
        Student s1 = new Student("Alice");
        Student s2 = new Student("Bob");
    }
}
```

```
        System.out.println(s1.name + " and " + s2.name);  
        // When no more references exist, GC can reclaim the memory  
        s1 = null;  
        s2 = null;  
        System.gc(); // suggestion only - JVM decides when to collect  
        System.out.println("Objects may be garbage-collected now");  
    }  
}
```

3. Program Lifecycle (Compilation & Execution)

A Java program goes through the following stages:

- **1. Write source code** — Create a .java file (e.g., Hello.java) using any text editor or IDE.
- **2. Compilation** — The javac compiler translates source code into platform-independent bytecode (.class file). Command: `javac Hello.java` → produces `Hello.class`
- **3. Loading** — Class Loader loads the .class file into JVM memory.
- **4. Linking** — Verification (bytecode safety), Preparation (allocate memory for static fields), Resolution (symbolic references).
- **5. Initialization** — Static variables and static blocks are executed.
- **6. Execution** — Execution Engine (interpreter + JIT) runs the bytecode. Command: `java Hello`
- **7. Garbage Collection** — Unused objects are automatically reclaimed during or after execution.

Note: From Java 11 onwards you can also run a single-file source-code program directly with `java Hello.java` (no explicit compilation step needed for simple cases).

Example 1: Full compile-and-run cycle demonstration

```
// File: LifecycleDemo.java
public class LifecycleDemo {
    // Static block - runs during class initialization
    static {
        System.out.println("1. Static block executed (class initialization)");
    }

    // Instance block
    {
        System.out.println("2. Instance block executed");
    }

    public LifecycleDemo() {
        System.out.println("3. Constructor executed");
    }

    public static void main(String[] args) {
        System.out.println("4. main() started");
        LifecycleDemo obj = new LifecycleDemo();
        System.out.println("5. main() finished");
    }
}
/*
Compile : javac LifecycleDemo.java
Run      : java LifecycleDemo
```

Output order:

1. Static block executed (class initialization)
 4. main() started
 2. Instance block executed
 3. Constructor executed
 5. main() finished
- */

Example 2: Demonstrating bytecode generation idea

```
public class SimpleCalc {
    public static int add(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        int result = add(10, 20);
        System.out.println("Sum = " + result);
    }
}
```

```
}  
// After compilation, SimpleCalc.class contains bytecode.  
// You can inspect it with: javap -c SimpleCalc
```

4. IDE Setup (Eclipse, IntelliJ)

Eclipse

Eclipse is a free, open-source IDE popular in academia and many enterprises. Download the “Eclipse IDE for Java Developers” package from eclipse.org. Key features: excellent refactoring, powerful debugger, plugin ecosystem (Marketplace), built-in Git support, and good Maven/Gradle integration. Workspace concept: projects live inside a workspace directory.

IntelliJ IDEA

IntelliJ IDEA (Community Edition is free) is widely regarded as the most intelligent Java IDE. Developed by JetBrains. Strengths: smart code completion, deep static analysis, excellent refactoring, built-in terminal, database tools, and outstanding support for Spring, Kotlin, etc. Ultimate Edition (paid) adds enterprise frameworks. Most modern companies prefer IntelliJ.

Common Setup Steps (both IDEs)

- Install a JDK first (Temurin, Oracle, Amazon Corretto, etc.) and set `JAVA_HOME`.
- Download and install the IDE.
- Configure the JDK inside the IDE (Project Structure / Installed JREs).
- Create a new Java Project → New Class with main method.
- Write code → Run (green play button) or Debug.
- Use built-in terminal or external terminal for `javac/java` if needed.

Example 1: Minimal program you would create in any IDE

```
public class IdeDemo {  
    public static void main(String[] args) {  
        System.out.println("Running from IDE");  
        for (int i = 1; i <= 5; i++) {  
            System.out.println("Count: " + i);  
        }  
    }  
}
```

Example 2: Using command-line tools (useful even with IDE)

```
// After writing Hello.java in any editor or IDE:  
// 1. Open terminal in the source folder  
// 2. Compile  
javac Hello.java  
// 3. Run  
java Hello  
// 4. (Optional) Create a JAR  
jar cfe Hello.jar Hello Hello.class  
java -jar Hello.jar
```

5. Variables (Declaration & Initialization)

A variable is a named memory location used to store data. In Java every variable must be declared with a type before it is used.

Declaration

Syntax: `type variableName;`

Example: `int age; String name;`

Initialization

Assigning a value for the first time: `age = 21;` or combined: `int age = 21;`

Types of Variables

Local variables — Declared inside a method, constructor or block. Must be initialized before use. No default value. Scope is the block in which they are declared.

Instance variables — Declared inside a class but outside any method. Belong to each object. Have default values (0, null, false...). Lifetime is the lifetime of the object.

Static (class) variables — Declared with the `static` keyword. Shared by all instances of the class. Belong to the class rather than any object.

Example 1: Local, Instance and Static variables

```
public class VariableDemo {
    // Instance variable
    int instanceCount = 0;

    // Static variable
    static int staticCount = 0;

    public void increment() {
        // Local variable
        int localTemp = 1;
        instanceCount += localTemp;
        staticCount += localTemp;
    }

    public static void main(String[] args) {
        VariableDemo v1 = new VariableDemo();
        VariableDemo v2 = new VariableDemo();

        v1.increment();
        v2.increment();
        v2.increment();

        System.out.println("v1.instanceCount = " + v1.instanceCount); // 1
        System.out.println("v2.instanceCount = " + v2.instanceCount); // 2
        System.out.println("staticCount      = " + VariableDemo.staticCount); // 3
    }
}
```

Example 2: Multiple declaration & initialization styles

```
public class DeclareInit {
    public static void main(String[] args) {
        // Separate declaration and initialization
        int a;
        a = 10;
    }
}
```

```
// Combined
int b = 20;

// Multiple variables of same type
int x = 1, y = 2, z = 3;

// Final variable (constant)
final double PI = 3.14159;

System.out.println("a=" + a + " b=" + b + " x+y+z=" + (x+y+z));
System.out.println("PI = " + PI);
    }
}
```

6. Data Types (Primitive & Wrapper)

Primitive Data Types

Java has 8 primitive types that store simple values directly:

Type	Size	Default	Range (approx)
byte	1 byte	0	-128 to 127
short	2 bytes	0	-32,768 to 32,767
int	4 bytes	0	-2 ³¹ to 2 ³¹ -1
long	8 bytes	0L	-2 ⁶³ to 2 ⁶³ -1
float	4 bytes	0.0f	~±3.4e38 (7 digits)
double	8 bytes	0.0d	~±1.8e308 (15 digits)
char	2 bytes	'\u0000'	0 to 65,535 (Unicode)
boolean	JVM dependent	false	true / false

Wrapper Classes

Each primitive has a corresponding wrapper class in java.lang: Byte, Short, Integer, Long, Float, Double, Character, Boolean. Wrappers allow primitives to be treated as objects (needed for Collections, Generics, etc.). Java supports **Autoboxing** (primitive → wrapper) and **Unboxing** (wrapper → primitive) automatically.

Example 1: Primitive types in action

```
public class PrimitiveDemo {
    public static void main(String[] args) {
        byte b = 100;
        short s = 30000;
        int i = 2_000_000_000;    // underscore for readability
        long l = 9_000_000_000L;  // L suffix required
        float f = 3.14f;         // f suffix required
        double d = 3.1415926535;
        char c = 'A';
        boolean flag = true;

        System.out.println("byte  = " + b);
        System.out.println("short = " + s);
        System.out.println("int   = " + i);
        System.out.println("long  = " + l);
        System.out.println("float = " + f);
        System.out.println("double= " + d);
        System.out.println("char  = " + c + " (code=" + (int)c + ")");
        System.out.println("boolean= " + flag);
    }
}
```

Example 2: Wrapper classes, Autoboxing & Unboxing

```
public class WrapperDemo {
    public static void main(String[] args) {
        // Explicit boxing
        Integer obj1 = Integer.valueOf(42);

        // Autoboxing
        Integer obj2 = 100;           // int → Integer automatically

        // Unboxing
    }
}
```

```
int primitive = obj2;          // Integer → int automatically

// Useful methods
System.out.println("parseInt : " + Integer.parseInt("123"));
System.out.println("max value: " + Integer.MAX_VALUE);
System.out.println("bit count: " + Integer.bitCount(15)); // 4

// Null is possible with wrappers (danger of NPE)
Integer nullable = null;
// int x = nullable; // NullPointerException at runtime
    }
}
```

7. Operators (Arithmetic, Relational, Logical)

Arithmetic Operators

+ (addition) – (subtraction) * (multiplication) / (division) % (modulus)

Also unary + and –, increment ++, decrement — (pre and post forms).

Relational (Comparison) Operators

== != > < >= <= All produce a boolean result.

Logical Operators

&& (logical AND – short-circuit) || (logical OR – short-circuit) ! (NOT)

Also bitwise & | ^ ~ and shift operators << >> >>> exist, but short-circuit logical operators are most commonly used in conditions.

Example 1: Arithmetic & Relational operators

```
public class ArithmeticRelational {
    public static void main(String[] args) {
        int a = 15, b = 4;

        System.out.println("a + b = " + (a + b)); // 19
        System.out.println("a - b = " + (a - b)); // 11
        System.out.println("a * b = " + (a * b)); // 60
        System.out.println("a / b = " + (a / b)); // 3 (integer division)
        System.out.println("a % b = " + (a % b)); // 3

        System.out.println("a > b = " + (a > b)); // true
        System.out.println("a == b = " + (a == b)); // false
        System.out.println("a != b = " + (a != b)); // true

        int x = 5;
        System.out.println("x++ = " + (x++)); // 5 (post-increment)
        System.out.println("x = " + x); // 6
        System.out.println("++x = " + (++x)); // 7 (pre-increment)
    }
}
```

Example 2: Logical operators & short-circuit behavior

```
public class LogicalDemo {
    static boolean sideEffect() {
        System.out.println("sideEffect() called");
        return true;
    }

    public static void main(String[] args) {
        int age = 20;
        boolean hasLicense = true;

        // Short-circuit AND
        if (age >= 18 && hasLicense) {
            System.out.println("Can drive");
        }

        // Short-circuit: second operand not evaluated if first is false
        boolean result = false && sideEffect(); // sideEffect NOT called
        System.out.println("result = " + result);

        // Non-short-circuit & would call sideEffect even if left is false
        result = false & sideEffect(); // sideEffect IS called
    }
}
```

```
        System.out.println("result = " + result);  
    }  
}
```

8. Type Casting (Widening, Narrowing)

Widening (Implicit / Automatic) Casting

Converting a smaller type to a larger type. Safe – no data loss. Performed automatically by the compiler.

byte → short → int → long → float → double

char → int → long → float → double

Narrowing (Explicit) Casting

Converting a larger type to a smaller type. May lose data or precision. Must be written explicitly with a cast operator: (targetType) value.

Example 1: Widening casting

```
public class WideningDemo {
    public static void main(String[] args) {
        int i = 100;
        long l = i;           // automatic widening
        float f = l;          // automatic widening
        double d = f;         // automatic widening

        System.out.println("int    : " + i);
        System.out.println("long   : " + l);
        System.out.println("float  : " + f);
        System.out.println("double : " + d);

        char ch = 'A';
        int code = ch;        // char → int
        System.out.println("char 'A' as int = " + code); // 65
    }
}
```

Example 2: Narrowing casting (possible data loss)

```
public class NarrowingDemo {
    public static void main(String[] args) {
        double d = 100.99;
        int i = (int) d;      // fractional part truncated
        System.out.println("double 100.99 → int = " + i); // 100

        long big = 300L;
        byte b = (byte) big;  // overflow - value wraps around
        System.out.println("long 300 → byte = " + b); // 44 (300-256)

        int num = 65;
        char ch = (char) num;
        System.out.println("int 65 → char = " + ch); // A
    }
}
```

9. Input Handling (Scanner, BufferedReader)

Scanner (java.util.Scanner)

Convenient high-level class for reading input from System.in, files, or strings. Provides methods: nextInt(), nextDouble(), next(), nextLine(), hasNextXxx(), etc. Easy to use but relatively slower than BufferedReader for large input (important in competitive programming).

BufferedReader (java.io.BufferedReader)

More efficient for reading large amounts of text. Usually wrapped around an InputStreamReader. Reads lines with readLine(). Parsing of numbers must be done manually (Integer.parseInt, etc.). Preferred when performance matters.

Example 1: Using Scanner

```
import java.util.Scanner;

public class ScannerDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = sc.nextLine();

        System.out.print("Enter your age: ");
        int age = sc.nextInt();

        System.out.print("Enter your marks: ");
        double marks = sc.nextDouble();

        System.out.println("Name : " + name);
        System.out.println("Age : " + age);
        System.out.println("Marks : " + marks);

        sc.close(); // good practice
    }
}
```

Example 2: Using BufferedReader

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;

public class BufferedReaderDemo {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(
            new InputStreamReader(System.in));

        System.out.print("Enter your name: ");
        String name = br.readLine();

        System.out.print("Enter your age: ");
        int age = Integer.parseInt(br.readLine());

        System.out.print("Enter two numbers (space separated): ");
        String[] parts = br.readLine().split(" ");
        int a = Integer.parseInt(parts[0]);
        int b = Integer.parseInt(parts[1]);

        System.out.println("Name : " + name);
        System.out.println("Age : " + age);
    }
}
```

```
        System.out.println("Sum  : " + (a + b));  
        br.close();  
    }  
}
```

10. Access Modifiers (public, private, protected, default)

Access modifiers control the visibility of classes, methods, and variables.

Modifier	Same Class	Same Package	Subclass	Everywhere
private	Yes	No	No	No
default (no modifier)	Yes	Yes	No*	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

** default is package-private; subclass in another package cannot access.*

Example 1: Demonstrating private, public and default

```
// File: AccessDemo.java
class Person {
    private String secret = "hidden";    // only inside Person
    String name = "John";                // default - same package
    public int age = 25;                  // accessible everywhere

    public String getSecret() {           // controlled access
        return secret;
    }
}

public class AccessDemo {
    public static void main(String[] args) {
        Person p = new Person();
        // System.out.println(p.secret); // COMPILER ERROR - private
        System.out.println(p.name);      // OK - same package
        System.out.println(p.age);       // OK - public
        System.out.println(p.getSecret()); // OK - via public method
    }
}
```

Example 2: protected access with inheritance

```
// Assume both classes in same package for simplicity
class Animal {
    protected String sound = "Some sound";
    protected void makeSound() {
        System.out.println(sound);
    }
}

class Dog extends Animal {
    public void bark() {
        sound = "Woof";    // protected member accessible in subclass
        makeSound();
    }
}

public class ProtectedDemo {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.bark();           // prints Woof
        // d.sound;         // also accessible if same package
    }
}
```

11. final Keyword (Variables, Methods, Classes)

The **final** keyword is used to restrict modification:

final variable — Once assigned, its value cannot be changed (constant). Must be initialized either at declaration or in constructor (for instance finals).

final method — Cannot be overridden by subclasses.

final class — Cannot be extended (no subclasses). Examples: String, Integer, Math.

Example 1: final variables and final method

```
class Calculator {
    final double PI = 3.14159;           // constant

    final int add(int a, int b) {        // cannot be overridden
        return a + b;
    }
}

// class AdvancedCalc extends Calculator {
//     int add(int a, int b) { ... }    // COMPILE ERROR
// }

public class FinalVarDemo {
    public static void main(String[] args) {
        Calculator c = new Calculator();
        System.out.println("PI = " + c.PI);
        System.out.println("Sum = " + c.add(5, 7));
        // c.PI = 3.14;                // COMPILE ERROR
    }
}
```

Example 2: final class

```
final class Constants {
    public static final int MAX_USERS = 1000;
    public static final String APP_NAME = "MyApp";
}

// class ExtendedConstants extends Constants { } // COMPILE ERROR

public class FinalClassDemo {
    public static void main(String[] args) {
        System.out.println(Constants.APP_NAME);
        System.out.println("Max users allowed: " + Constants.MAX_USERS);
    }
}
```

12. Object Class (toString(), equals())

Every class in Java implicitly extends **java.lang.Object**. Important methods that are frequently overridden:

toString() — Returns a string representation of the object. Default implementation returns `ClassName@hashCode`. Override it for meaningful output.

equals(Object obj) — Compares two objects for equality. Default implementation compares references (`==`). Override it to compare content (and also override `hashCode()` to maintain the contract).

Example 1: Overriding toString()

```
class Student {
    String name;
    int roll;

    Student(String name, int roll) {
        this.name = name;
        this.roll = roll;
    }

    @Override
    public String toString() {
        return "Student[name=" + name + ", roll=" + roll + "]";
    }
}

public class ToStringDemo {
    public static void main(String[] args) {
        Student s = new Student("Alice", 101);
        System.out.println(s);           // calls toString() automatically
        System.out.println(s.toString());
    }
}

// Output: Student[name=Alice, roll=101]
```

Example 2: Overriding equals() and hashCode()

```
class Student {
    String name;
    int roll;

    Student(String name, int roll) {
        this.name = name;
        this.roll = roll;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        Student other = (Student) obj;
        return roll == other.roll && name.equals(other.name);
    }

    @Override
    public int hashCode() {
        return name.hashCode() * 31 + roll;
    }
}

public class EqualsDemo {
    public static void main(String[] args) {
        Student s1 = new Student("Bob", 102);
        Student s2 = new Student("Bob", 102);
    }
}
```

```
        Student s3 = new Student("Carol", 103);

        System.out.println(s1.equals(s2)); // true  (content equal)
        System.out.println(s1.equals(s3)); // false
        System.out.println(s1 == s2);      // false (different references)
    }
}
```

13. if-else (Nested Conditions)

The if-else statement executes a block of code based on a boolean condition. Nested if-else statements allow multi-level decision making.

Syntax forms: if, if-else, if-else-if ladder, nested if.

Example 1: if-else-if ladder (grade calculation)

```
public class GradeCalculator {
    public static void main(String[] args) {
        int marks = 78;

        if (marks >= 90) {
            System.out.println("Grade: A+");
        } else if (marks >= 80) {
            System.out.println("Grade: A");
        } else if (marks >= 70) {
            System.out.println("Grade: B");
        } else if (marks >= 60) {
            System.out.println("Grade: C");
        } else if (marks >= 50) {
            System.out.println("Grade: D");
        } else {
            System.out.println("Grade: F (Fail)");
        }
    }
}
// Output: Grade: B
```

Example 2: Nested if-else (largest of three numbers)

```
public class LargestOfThree {
    public static void main(String[] args) {
        int a = 25, b = 40, c = 15;

        if (a >= b) {
            if (a >= c) {
                System.out.println(a + " is the largest");
            } else {
                System.out.println(c + " is the largest");
            }
        } else {
            if (b >= c) {
                System.out.println(b + " is the largest");
            } else {
                System.out.println(c + " is the largest");
            }
        }
    }
}
// Output: 40 is the largest
```

14. switch (Traditional & Enhanced)

Traditional switch (before Java 14)

Works with byte, short, char, int, enum, String (from Java 7), and a few wrapper classes. Requires break statements to prevent fall-through.

Enhanced switch (Java 14+)

Introduces arrow labels (->) that do not fall through, multi-constant case labels, and the ability to return a value (switch expression).

Example 1: Traditional switch

```
public class TraditionalSwitch {
    public static void main(String[] args) {
        int day = 3;
        String dayName;

        switch (day) {
            case 1:
                dayName = "Monday";
                break;
            case 2:
                dayName = "Tuesday";
                break;
            case 3:
                dayName = "Wednesday";
                break;
            case 4:
                dayName = "Thursday";
                break;
            case 5:
                dayName = "Friday";
                break;
            case 6:
                dayName = "Saturday";
                break;
            case 7:
                dayName = "Sunday";
                break;
            default:
                dayName = "Invalid day";
        }
        System.out.println(dayName); // Wednesday
    }
}
```

Example 2: Enhanced switch (expression + arrow)

```
public class EnhancedSwitch {
    public static void main(String[] args) {
        int day = 6;

        // Switch expression returning a value
        String type = switch (day) {
            case 1, 2, 3, 4, 5 -> "Weekday";
            case 6, 7         -> "Weekend";
            default            -> "Invalid";
        };
        System.out.println(type); // Weekend

        // Arrow form with block
        switch (day) {
```

```
        case 1, 2, 3, 4, 5 -> System.out.println("Work hard");
        case 6, 7         -> {
            System.out.println("Relax");
            System.out.println("Enjoy");
        }
        default           -> System.out.println("Error");
    }
}
```

15. for Loop (Iteration)

The for loop is used when the number of iterations is known in advance. Classic form: for (initialization; condition; update)

Also supports the enhanced for-each loop for arrays and Iterable collections.

Example 1: Classic for loop – printing a pattern

```
public class ForLoopPattern {
    public static void main(String[] args) {
        // Print a right-angled triangle of stars
        for (int i = 1; i <= 5; i++) {
            for (int j = 1; j <= i; j++) {
                System.out.print("* ");
            }
            System.out.println();
        }
    }
}
/*
Output:
*
* *
* * *
* * * *
* * * * *
*/
```

Example 2: Enhanced for-each loop + classic sum

```
public class ForEachDemo {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40, 50};

        // Enhanced for-each
        System.out.print("Elements: ");
        for (int n : numbers) {
            System.out.print(n + " ");
        }
        System.out.println();

        // Classic for - calculate sum
        int sum = 0;
        for (int i = 0; i < numbers.length; i++) {
            sum += numbers[i];
        }
        System.out.println("Sum = " + sum); // 150
    }
}
```

16. while Loop (Entry Controlled)

The while loop is an **entry-controlled** loop: the condition is checked *before* each iteration. If the condition is false at the start, the body never executes. Suitable when the number of iterations is not known in advance.

Example 1: Simple countdown

```
public class WhileCountdown {
    public static void main(String[] args) {
        int n = 5;
        while (n > 0) {
            System.out.println("T-minus " + n);
            n--;
        }
        System.out.println("Lift off!");
    }
}
```

```
/*
```

Output:

```
T-minus 5
T-minus 4
T-minus 3
T-minus 2
T-minus 1
Lift off!
```

```
*/
```

Example 2: Reading until a sentinel value (concept)

```
import java.util.Scanner;

public class WhileSentinel {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int sum = 0;
        System.out.println("Enter numbers (0 to stop):");

        int num = sc.nextInt();          // read first value
        while (num != 0) {                // entry-controlled
            sum += num;
            num = sc.nextInt();           // read next
        }
        System.out.println("Sum = " + sum);
        sc.close();
    }
}
```

17. do-while Loop (Exit Controlled)

The do-while loop is an **exit-controlled** loop: the body is executed at least once, and the condition is checked *after* each iteration. Useful for menu-driven programs or any situation where you want the body to run before testing the condition.

Example 1: Simple do-while

```
public class DowhileDemo {
    public static void main(String[] args) {
        int i = 1;
        do {
            System.out.println("Iteration " + i);
            i++;
        } while (i <= 3);

        // Condition false from the beginning - still runs once
        int j = 10;
        do {
            System.out.println("This prints once even though j > 5");
            j++;
        } while (j <= 5);
    }
}
```

Example 2: Menu-driven program skeleton

```
import java.util.Scanner;

public class MenuDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int choice;

        do {
            System.out.println("\n===== MENU =====");
            System.out.println("1. Say Hello");
            System.out.println("2. Show Date info");
            System.out.println("3. Exit");
            System.out.print("Enter choice: ");
            choice = sc.nextInt();

            switch (choice) {
                case 1 -> System.out.println("Hello, User!");
                case 2 -> System.out.println("Today is a great day to code.");
                case 3 -> System.out.println("Goodbye!");
                default -> System.out.println("Invalid choice");
            }
        } while (choice != 3);

        sc.close();
    }
}
```

18. break & continue (Flow Control)

break — Immediately terminates the nearest enclosing loop or switch statement. Control passes to the statement after the loop/switch.

continue — Skips the rest of the current iteration and proceeds with the next iteration of the nearest enclosing loop.

Example 1: break – finding first even number

```
public class BreakDemo {
    public static void main(String[] args) {
        int[] arr = {3, 7, 9, 12, 15, 18};

        for (int n : arr) {
            if (n % 2 == 0) {
                System.out.println("First even number: " + n);
                break;    // exit the loop immediately
            }
        }
        System.out.println("Search finished");
    }
}
// Output:
// First even number: 12
// Search finished
```

Example 2: continue – print only odd numbers

```
public class ContinueDemo {
    public static void main(String[] args) {
        System.out.print("Odd numbers from 1 to 10: ");
        for (int i = 1; i <= 10; i++) {
            if (i % 2 == 0) {
                continue;    // skip even numbers
            }
            System.out.print(i + " ");
        }
        System.out.println();
    }
}
// Output: Odd numbers from 1 to 10: 1 3 5 7 9
```

19. Nested Loops (Dry Run)

A nested loop is a loop placed inside the body of another loop. The inner loop executes completely for each iteration of the outer loop. Nested loops are essential for 2-D arrays, patterns, matrix operations, and many algorithmic problems. **Dry-run** means manually tracing the values of loop variables step-by-step on paper to understand the flow and verify correctness.

Example 1: Multiplication table (nested for) + dry-run idea

```
public class NestedMultiplication {
    public static void main(String[] args) {
        for (int i = 1; i <= 3; i++) {           // outer
            for (int j = 1; j <= 3; j++) {       // inner
                System.out.print(i * j + "\t");
            }
            System.out.println();
        }
    }
}
```

```
/*
Dry-run (i, j → product):
i=1, j=1 → 1
i=1, j=2 → 2
i=1, j=3 → 3    then newline
i=2, j=1 → 2
i=2, j=2 → 4
i=2, j=3 → 6    then newline
i=3, j=1 → 3
i=3, j=2 → 6
i=3, j=3 → 9
*/
```

Output:

```
1  2  3
2  4  6
3  6  9
*/
```

Example 2: Pattern printing with dry-run comments

```
public class NestedPattern {
    public static void main(String[] args) {
        int n = 4;
        // Pyramid of numbers
        for (int i = 1; i <= n; i++) {
            // spaces
            for (int s = 1; s <= n - i; s++) {
                System.out.print(" ");
            }
            // increasing numbers
            for (int j = 1; j <= i; j++) {
                System.out.print(j + " ");
            }
            // decreasing numbers
            for (int j = i - 1; j >= 1; j--) {
                System.out.print(j + " ");
            }
            System.out.println();
        }
    }
}
```

```
/*
Dry-run for n=4 (spaces shown as .):
i=1: . . . 1
i=2: . . 1 2 1
i=3: . 1 2 3 2 1
*/
```

```
i=4: 1 2 3 4 3 2 1
```

Actual output:

```
      1
    1 2 1
  1 2 3 2 1
1 2 3 4 3 2 1
*/
```

End of Guide

You now have complete conceptual coverage and two practical programs for every topic listed in your placement syllabus. Practice typing the programs yourself, dry-run them, modify them, and solve related problems on platforms such as LeetCode, HackerRank, or GeeksforGeeks.

Suggested next steps:

- Arrays & Strings
- OOP pillars (Encapsulation, Inheritance, Polymorphism, Abstraction)
- Exception Handling
- Collections Framework
- Multithreading basics
- Common interview coding patterns

All the best for your placements!